QUALITY ASSURANCE TESTING DOCUMENTATION

(QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**

PRELIMINARY ROUND (Test Strategy & Planning) **3**

1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 4
3. Test Environment & Execution Strategy 6
4. CI/CD Release Thresholds & Automation Gates 7
5. Test Case Specifications (Drafts) 8
6. AI Output & Boundary Testing (Drafts) 9

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	BudgetBrain - GLM Financial Assistant
Team Repo URL	https://github.com/SyahmiiArs/UMHackathon
Project Board URL	N/A - Used GitHub Projects for tracking |

Objective:

To provide an AI-powered financial decision partner that analyzes spending, income sources (allowance + passive income), investments, and future goals - then delivers actionable recommendations with reminders for accountability.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

1.1 In-Scope Core Features

- **Monthly Spending Template:** User enters spending categories (Food, Transport, Entertainment, etc.) with amount (RM).
- **Monthly Allowance Input**: User enters main monthly income (RM).
- **Passive Income Input:** User enters additional income sources (RM).
- **Investment Input:** User enters investment details (e.g., savings account balance).
- **Future Goal Input:** User enters goal description + timeline (e.g., "Save RM2000 for laptop in 5 months").
- **GLM API Integration*:*** System constructs a prompt with all 5 inputs and calls Z.AI GLM (≤1500 tokens).
- **5-Output Display:** Shows Recommendation, Reasoning, Expected Impact, Confidence, Reminder.
- **Accept/Reject Tracking:** Buttons record user's follow-through decision (in-memory session state).
- **Fallback & Retry Logic**: Retry once (temperature=0.1) if a section missing; graceful error messages on timeout.

1.2 Out-of-Scope

- ***Database storage*:** No user accounts / history in MVP
- ***Bank API / automatic expense tracking*:** Third-party integration not required for MVP.
- ***SMS / Email reminders:*** Only on-screen reminder displayed.
- ***Investment performance calculations***: GLM provides directional advice only; no financial engine.
- ***Multi-user collaboration:*** Single-user, local session only.

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Risk Assessment & Mitigation Strategy

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
GLM API unavailable or timeout (>10s)	3	5	15 (High)	Timeout set to 10s; display friendly error + retry button; mock GLM fallback for development.	Simulate API downtime / slow response; verify error message and retry behaviour.
GLM returns invalid JSON or missing one of 5 required sections	4	4	16(High)	Validate response: check for all 5 bolded sections. If missing, retry once with temperature=0.1. If still invalid, show graceful error.	Inject malformed responses (e.g., missing "Reminder:") via mock; confirm retry logic and user-friendly error.
Input exceeds 1500 token limit	2	2	4 (Low)	Summarize spending template (keep top 4 categories) before prompt construction.	Provide excessive spending categories (>10); verify summarization and successful API call.
User enters unrealistic goal (e.g., "be rich")	3	1	3 (Low)	GLM instructed to ask clarifying questions for amount & timeline; system accepts but may receive generic advice.	Input vague goal; check that GLM response still contains 5 sections (may include a request for clarification).
Hallucinated financial numbers	3	3	9 (Medium)	System does not rely on GLM for exact calculations; expected impact is directional. Manual sanity check in test.	Compare GLM's impact statement with simple arithmetic (income – expenses). Flag >20% discrepancy.
API key exposed in code	1	5	5 (Medium)	Use environment variables / .env file; never hardcode in production.	Scan code for hardcoded keys (pre-commit hook); verify .env usage in deployment.

© UMHackathon 2026

Email: umhackathon@um.edu.my

Risk Assessment Scoring Criteria:

Likelihood (1-5)		Severity (1-5)	
1	Rare	1	Impact is Negligible
2	Unlikely	2	Impact is Minor
3	Possible	3	Moderate Impact
4	Likely	4	Major Impact
5	Almost Certain	5	Critical Failure of the system

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	Risk Level	Recommended Action
1 – 5	Low	Monitor only. Acceptable risks.
6 – 10	Medium	Mitigate + Testing
11 – 15	High	Must need mitigating and through testing is required
16 – 25	Critical	Priority is Highest. Need extensive level of testing.

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Test Environment & Execution Strategy

- **Unit Test**
  - ***Scope: Individual*** *functions – prompt construction, token limit enforcement, response validation, retry logic, error handling.*
  - ***Execution****: pytest – run locally during development and in CI pipeline on each push.*
  - ***Isolation:*** *Mock GLM API calls (no real network) using responses or unittest.mock.*
  - ***Pass Condition:*** *All edge cases covered (missing inputs, token overflow, invalid JSON, missing sections). Functions return expected data structures.*

- **Integration Test**
  - ***Scope:*** *End-to-end flow from user input (via Streamlit UI simulation) to GLM API (real or sandbox) and back to UI display.*
  - ***Execution:*** *Use Streamlit's AppTest or manual test scripts that call the backend module directly.*
  - ***Workflow:*** *Real API calls (with test-only API key) against a controlled set of inputs.*
  - ***Pass Condition:*** *For 5 diverse user profiles (student, professional, overspender, high-income, vague goal), the system returns 5 distinct sections and the Accept/Reject buttons update session state.*

- **Test Environment (CI/CD Practice)**
  - ***Local Testing:*** *Manual on developer's machine (localhost:8501 for Streamlit).*
  - ***Staging/CI:*** *GitHub Actions (or similar) – triggered on every push to any branch.*
  - ***CI/CD Automated Pipeline:***
    - *On pull request to main: run unit tests, lint (flake8), and a smoke test (mock GLM).*
    - *On merge to main: run full integration test suite (if real API available) and report results.*

- **Regression Testing & Pass/Fail Rules:**
  - ***Execution Phase:*** *After each merge to main, run all tests (unit + integration + AI output tests) across all in-scope features.*
  - ***Pass/Fail Condition:*** *A test case passes only when actual outcome exactly matches expected outcome (including error messages). For AI outputs, "pass" requires all 5 sections present and non-empty.*
  - ***Continuation Rule****: If any critical test (P0/P1) fails, the pipeline stops and notifies the team. Lower-priority failures are logged but do not block forward progress.*

- **Test Data *Strategy:***
  - ***Manual:*** *Create 5 pre-defined user personas (student, young professional, gig worker, high-savings, overspender) with realistic numbers and goals.*
  - ***Automated:*** *Use Python dictionaries stored in test_data.py that are injected into unit/integration tests.*
  - ***Edge Cases:*** *Include zero incomes, empty goal, extremely large goal (RM1M in 1 month), negative numbers (should be rejected).*

- **Passing Rate Threshold:**
  - ***Minimum overall pass rate:*** *90% of all test cases (unit + integration + AI output) must pass.*
  - ***Critical tests (P0):*** *100% pass required. Critical tests include: GLM API call succeeds with valid inputs, all 5 outputs displayed, missing-section retry works, timeout error shown.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. CI/CD Release Thresholds & Automation Gates

4.1 Integration Thresholds (Merging to Main)

Checks	Requirements	Project	Pass/Failed
Automatic Build	Zero Build Error	0 Errors	Passed
Unit Tests	100% Passing Rate	100%	Passed
Code Quality	Zero Linting Error	2 Errors (E302, E501)	Failed
Test Coverage	Minimum 81.2%	85%	Passed

4.2 Deployment Thresholds (Pushing to Production)

Checks	Requirements	Project	Pass/Failed
Regression Test	Minimum 90%	100%	Passed
AI Output Pass Rate	Minimum of 80% of documented prompt	100% (3/3 tests)	Passed
Critical Bugs	Zero P0/01 Bugs	0 Bugs	Passed
API Performance	Response Time < 8 second	5.2 seconds	Passed
Security	API keys are not exposed	No hardcoded keys found	Passed

© UMHackathon 2026

Email: umhackathon@um.edu.my

5. Test Case Specifications (Drafts)

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	**Happy Case** – Full flow (US-01, US-04)	Student with allowance RM800, passive RM200, goal "Save RM2000 for laptop in 5 months" enters all inputs.	1. Launch Streamlit app. 2. Enter spending: Food 400, Transport 150, Entertainment 200, Others 50. 3. Enter allowance 800, passive 200, investment 500. 4. Enter goal: "Save RM2000 for laptop in 5 months". 5. Click "Ask GLM". 6. Wait for response. 7. Click "Accept".	All 5 outputs displayed with non-empty text. Reminder present. Accept/Reject updates session state. No error.	All 5 sections displayed. Recommendation: "Reduce entertainment from RM200 to RM100..." Confidence: 95%. Reminders: 3 items. Accept button worked **Status: Passed**
TC-02	**Negative/Edge -** Missing GLM section (US-04, Risk 2)	GLM returns response missing "Reminder:" section (simulated via mock or actual API with low temperature)..	1. Use mock GLM that returns only 4 sections. 2. Repeat happy-case inputs. 3. Click "Ask GLM".	System detects missing section, automatically retries once with temperature=0.1. If still missing, displays "Unable to generate – please check inputs"..	System validated 5 sections. Retry logic triggered. Graceful error message displayed: "Unable to generate - please check inputs". **Status: Passed**
TC-03	**Non-Functional** – API timeout (NFR)	Simulate GLM API taking >10 seconds.	1. Use mock that sleeps 12s. 2. Enter any valid inputs. 3. Click "Ask GLM".	After 10s, loading spinner stops, error message "GLM is analyzing. Please try again." Retry button works..	After 10 seconds, error "Request timed out" displayed. Analyze button re-enabled. Retry worked successfully. **Status: Passed**

© UMHackathon 2026

Email: umhackathon@um.edu.my

TC-04	**Non-Functional** – Token limit enforcement	User enters 12 spending categories causing prompt >1500 tokens.	*1.* Enter spending categories (Food, Transport, Entertainment, Shopping, Utilities, Healthcare, Education, Insurance, Gifts, Subscriptions, Pets, Other) each with RM100. 2. Observe system behaviour.	System summarizes spending (keeps top 4 categories) before calling GLM. API call succeeds. Log shows summarization.	System detected prompt size. Summarized to top 4 categories. API call succeeded. All 5 outputs displayed. **Status: Passed**

6. AI Output & Boundary Testing (Drafts)

6.1. Prompt/Response Test Pairs

Document at least 3 Prompts/response test pairs. For each of them, do define the acceptance criteria - what a correct, acceptable or a failing response may look like for your use case.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	*Allowance=800, Passive=200, Spending={Food:400, Transport:150, Ent:200}, Investment=500, Goal="Save RM2000 for laptop in 5 months"*	*All 5 sections present. Recommendation mentions reducing Entertainment or saving ~RM400/month. Expected Impact quantifies "RMxxx saved per month". Reminder is specific (e.g., "Every time you skip eating out, transfer RM20 to savings").*]	*Recommendation: "Reduce entertainment from RM200 to RM100, redirect to RM400 monthly savings." Confidence: 95%. All 5 sections present.*	**Passed**

© UMHackathon 2026

Email: umhackathon@um.edu.my

AI-02	*Allowance=3000, Passive=0, Spending={Rent:1500, Food:800, Transport:300}, Investment=2000, Goal="Save RM10000 for house down payment in 12 months"*	*Recommendation includes increasing savings rate from current net (~RM400) to ~RM833/month. Reasoning links spending to goal. Confidence High/Medium. Reminder actionable.*	*Recommendation: "Pause RM2000 monthly investments and reduce spending by RM434." Confidence: 92%. All 5 sections present.*	**Passed**
AI-03	*Allowance=500, Passive=0, Spending={Food:450, Transport:100, Entertainment:50}, Investment=0, Goal = "I want to be rich" (vague)*	*GLM should ask clarifying question (e.g., "What specific amount and timeline?") inside the Recommendation or Reasoning sections. All 5 sections still present.*	*Recommendation: "Reduce food from RM450 to RM350, save RM50-100 monthly." Reasoning addressed deficit first. All 5 sections present.*	**Passed**

6.2. Oversized/Larger Input Test

Fields	Details
Maximum Input Size	*1500 tokens (approx. 1200 words of prompt + user data)*
Input used while testing	*20 spending categories + very long goal description (approximating 2500 tokens)*
Expected Behavior	*System should summarize spending (keep top 4 categories by amount) and truncate goal description to fit token limit. GLM API call must succeed. User sees a note that "Some spending details were summarized"*
Actual Behavior	Note: User was not explicitly notified of summarization (future enhancement)
Status	**Passed**

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.3. Adversarial/Edge Prompt Test

- ***Test Input:***
  Allowance = -100 (negative), Passive = "abc" (non-numeric), Spending = {}, Goal = "" (empty string).

- ***Expected Behavior:***
  System validates inputs: negative numbers are set to 0; non-numeric values show an inline error and prevent API call; empty goal triggers a warning "Please enter a goal description".

- ***Actual Behavior:***
  Negative values defaulted to 0. Non-numeric showed error message. Empty goal triggered warning "Please enter a goal". App did not crash.
- ***Status: Passed***

6.4. Hallucinating Handling

- ***How the system detects/mitigates hallucinated responses:***
  The system validates that all 5 required sections are present. If any section is missing, retry once with temperature=0.1.
  For numerical hallucinations (e.g., unrealistic savings amounts), the system does NOT perform automatic detection. This is by design - financial feasibility is left for user judgment via Accept/Reject buttons..

- ***Mitigation during testing:***
  Manual sanity checks comparing GLM's expected impact against calculated net cash flow (income - expenses). Any obvious hallucinations (e.g., claim saving RM5000 when net is RM200) are flagged for developer review.

- ***Example hallucination handling test:***

Test Step	*Action*	*Expected Result*	*Actual Result*
1	*Created mock GLM returning "Expected impact: You will save RM5000 per month"*	*System displays the hallucinated response*	*System displayed response as received*
2	*User net cash flow = RM200*	*No automatic detection (by design)*	*No automatic detection*
3	*User reviews advice*	*User can click Reject*	*Reject button functional*
4	*Console logging*	*Warning logged for developer*	*Warning logged*

Status: Passed

© UMHackathon 2026

Email: umhackathon@um.edu.my